Algorithms and Data Structures

Queues, Linked Lists, Sorting, and Complexity

Martin Benning

University College London
ENGF0034 – Design and Professional Skills

20 October 2026



Lecture Overview

Learning Objectives

By the end of this lecture, you will be able to:

1. Distinguish **Abstract Data Types (ADTs)** from concrete **Data Structures**, and contrast **contiguous** vs. **linked** memory.
2. Implement an $\mathcal{O}(1)$ **FIFO Queue** using a singly-linked list (`simple_node.py`, `custom_queue.py`) and use `collections.deque`.
3. Formulate the **Sorting Problem** (including as a linear program) and prove the $\Omega(N \log N)$ comparison lower bound.
4. Analyse **Merge Sort** (`merge.py`), its stability, recurrence tree, higher-order comparators, `lambda`/`key` functions, and **Timsort**.

Lecture Outline

ADTs, Data Structures, and Complexity
The Queue ADT and Implementations
The Sorting Problem and Complexity
Merge Sort: Divide and Conquer
Generalisation, Higher-Order Functions, and Timsort
Summary

Lecture Overview

Learning Objectives

By the end of this lecture, you will be able to:

1. Distinguish **Abstract Data Types (ADTs)** from concrete **Data Structures**, and contrast **contiguous** vs. **linked** memory.
2. Implement an $\mathcal{O}(1)$ **FIFO Queue** using a singly-linked list (`simple_node.py`, `custom_queue.py`) and use `collections.deque`.
3. Formulate the **Sorting Problem** (including as a linear program) and prove the $\Omega(n \log n)$ comparison lower bound.
4. Analyse **Merge Sort** (`merge.py`), its stability, recurrence tree, higher-order comparators, `lambda`/`key` functions, and **Timsort**.

Lecture Outline

ADTs, Data Structures, and Complexity
The Queue ADT and Implementations
The Sorting Problem and Complexity
Merge Sort: Divide and Conquer
Generalisation, Higher-Order Functions, and Timsort
Summary

Lecture Overview

Learning Objectives

By the end of this lecture, you will be able to:

1. Distinguish **Abstract Data Types (ADTs)** from concrete **Data Structures**, and contrast **contiguous** vs. **linked** memory.
2. Implement an $\mathcal{O}(1)$ **FIFO Queue** using a singly-linked list (`simple_node.py`, `custom_queue.py`) and use `collections.deque`.
3. Formulate the **Sorting Problem** (including as a linear program) and prove the $\Omega(\mathrm{N} \log \mathrm{N})$ comparison lower bound.
4. Analyse **Merge Sort** (`merge.py`), its stability, recurrence tree, higher-order comparators, `lambda`/`key` functions, and **Timsort**.

Lecture Outline

ADTs, Data Structures, and Complexity
The Queue ADT and Implementations
The Sorting Problem and Complexity
Merge Sort: Divide and Conquer
Generalisation, Higher-Order Functions, and Timsort
Summary

Lecture Overview

Learning Objectives

By the end of this lecture, you will be able to:

1. Distinguish **Abstract Data Types (ADTs)** from concrete **Data Structures**, and contrast **contiguous** vs. **linked** memory.
2. Implement an $\mathcal{O}(1)$ **FIFO Queue** using a singly-linked list (`simple_node.py`, `custom_queue.py`) and use `collections.deque`.
3. Formulate the **Sorting Problem** (including as a linear program) and prove the $\Omega(\mathrm{N} \log \mathrm{N})$ comparison lower bound.
4. Analyse **Merge Sort** (`merge.py`), its stability, recurrence tree, higher-order comparators, `lambda`/`key` functions, and **Timsort**.

Lecture Outline

ADTs, Data Structures, and Complexity
The Queue ADT and Implementations
The Sorting Problem and Complexity
Merge Sort: Divide and Conquer
Generalisation, Higher-Order Functions, and Timsort
Summary

ADTs, Data Structures, and Complexity

Organising Data: Abstraction vs. Implementation

Abstract Data Type (ADT) — The "What"

An **ADT** is a mathematical model of a data type defined purely by its **interface** and **semantics** (behaviour) from the user's perspective:

- Specifies **what** operations exist and their expected behaviour, *not* how they are coded.
- **Examples:** Sequence/List, Queue, Stack, Set, Map.

Data Structure — The "How"

A **Data Structure** is a concrete representation of data in physical computer memory that implements an ADT:

- Specifies **how** memory is laid out and how operations execute step-by-step.
- **Examples:** Dynamic Array (`list`), Linked List, Hash Table.

Why Abstraction Matters (Week 1 Principle)

Separating the ADT interface from the underlying data structure lets us swap a slow implementation for a fast one **without changing a single line of client code**.

Organising Data: Abstraction vs. Implementation

Abstract Data Type (ADT) — The "What"

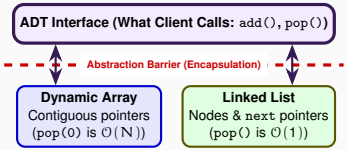
An **ADT** is a mathematical model of a data type defined purely by its **interface** and **semantics** (behaviour) from the user's perspective:

- Specifies **what** operations exist and their expected behaviour, *not* how they are coded.
- **Examples:** Sequence/List, Queue, Stack, Set, Map.

Data Structure — The "How"

A **Data Structure** is a concrete representation of data in physical computer memory that implements an ADT:

- Specifies **how** memory is laid out and how operations execute step-by-step.
- **Examples:** Dynamic Array (`list`), Linked List, Hash Table.



Why Abstraction Matters (Week 1 Principle)

Separating the ADT interface from the underlying data structure lets us swap a slow implementation for a fast one **without changing a single line of client code**.

Organising Data: Abstraction vs. Implementation

Abstract Data Type (ADT) — The "What"

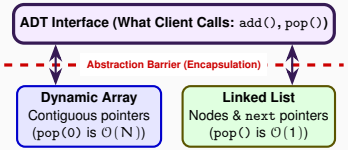
An **ADT** is a mathematical model of a data type defined purely by its **interface** and **semantics** (behaviour) from the user's perspective:

- Specifies **what** operations exist and their expected behaviour, *not* how they are coded.
- **Examples:** Sequence/List, Queue, Stack, Set, Map.

Data Structure — The "How"

A **Data Structure** is a concrete representation of data in physical computer memory that implements an ADT:

- Specifies **how** memory is laid out and how operations execute step-by-step.
- **Examples:** Dynamic Array (`list`), Linked List, Hash Table.



Why Abstraction Matters (Week 1 Principle)

Separating the ADT interface from the underlying data structure lets us swap a slow implementation for a fast one **without changing a single line of client code**.

Asymptotic Complexity: Upper ($\mathcal{O}$) vs. Lower (Ω) Bounds

Defining the Mathematical Symbols

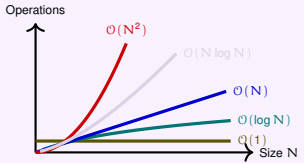
Let $T(N)$ be the operation count for input size $N \in \mathbb{N} = \{1, 2, 3, \dots\}$:

- **Big-$\mathcal{O}$ Upper Bound ($\mathcal{O}(g(N))$ — "at most"):** $T(N) \leqslant c \cdot g(N)$ for large N. Bounds the worst-case cost of a **specific algorithm**.
- **Big-Ω Lower Bound ($\Omega(g(N))$ — "at least"):** $T(N) \geqslant c \cdot g(N)$ for large N. Proves the minimum work required by **any algorithm** solving a problem.

New Growth Rate This Week: $\mathcal{O}(N \log N)$

Log-linear ($\mathcal{O}(N \log N)$): $\mathcal{O}(\log N)$ levels of work across N items. Far closer to linear $\mathcal{O}(N)$ than quadratic $\mathcal{O}(N^2)$ (for $N = 10^6$, $N \log_2 N \approx 2 \times 10^7$ vs. $N^2 = 10^{12}$!).

Complexity Hierarchy



Class	Name	Example
$\mathcal{O}(1)$	Constant	Array index / Queue pop
$\mathcal{O}(\log N)$	Logarithmic	Binary Search (Week 2)
$\mathcal{O}(N)$	Linear	Two-pointer Merge
$\mathcal{O}(N \log N)$	**Log-linear**	**Merge Sort / Timsort**
$\mathcal{O}(N^2)$	Quadratic	Bubble / Insertion Sort

Asymptotic Complexity: Upper ($\mathcal{O}$) vs. Lower (Ω) Bounds

Defining the Mathematical Symbols

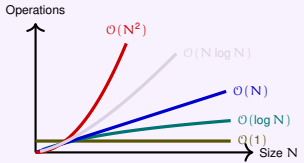
Let $T(N)$ be the operation count for input size $N \in \mathbb{N} = \{1, 2, 3, \dots\}$:

- **Big-$\mathcal{O}$ Upper Bound ($\mathcal{O}(g(N))$ — "at most"):** $T(N) \leqslant c \cdot g(N)$ for large N. Bounds the worst-case cost of a **specific algorithm**.
- **Big-Ω Lower Bound ($\Omega(g(N))$ — "at least"):** $T(N) \geqslant c \cdot g(N)$ for large N. Proves the minimum work required by **any algorithm** solving a problem.

New Growth Rate This Week: $\mathcal{O}(N \log N)$

Log-linear ($\mathcal{O}(N \log N)$): $\mathcal{O}(\log N)$ levels of work across N items. Far closer to linear $\mathcal{O}(N)$ than quadratic $\mathcal{O}(N^2)$ (for $N = 10^6$, $N \log_2 N \approx 2 \times 10^7$ vs. $N^2 = 10^{12}$!).

Complexity Hierarchy



Class	Name	Example
$\mathcal{O}(1)$	Constant	Array index / Queue pop
$\mathcal{O}(\log N)$	Logarithmic	Binary Search (Week 2)
$\mathcal{O}(N)$	Linear	Two-pointer Merge
$\mathcal{O}(N \log N)$	**Log-linear**	**Merge Sort / Timsort**
$\mathcal{O}(N^2)$	Quadratic	Bubble / Insertion Sort

Asymptotic Complexity: Upper ($\mathcal{O}$) vs. Lower (Ω) Bounds

Defining the Mathematical Symbols

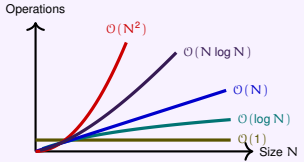
Let $T(N)$ be the operation count for input size $N \in \mathbb{N} = \{1, 2, 3, \dots\}$:

- **Big-$\mathcal{O}$ Upper Bound ($\mathcal{O}(g(N))$ — "at most"):** $T(N) \leqslant c \cdot g(N)$ for large N. Bounds the worst-case cost of a **specific algorithm**.
- **Big-Ω Lower Bound ($\Omega(g(N))$ — "at least"):** $T(N) \geqslant c \cdot g(N)$ for large N. Proves the minimum work required by **any algorithm** solving a problem.

New Growth Rate This Week: $\mathcal{O}(N \log N)$

Log-linear ($\mathcal{O}(N \log N)$): $\mathcal{O}(\log N)$ levels of work across N items. Far closer to linear $\mathcal{O}(N)$ than quadratic $\mathcal{O}(N^2)$ (for $N = 10^6$, $N \log_2 N \approx 2 \times 10^7$ vs. $N^2 = 10^{12}$!).

Complexity Hierarchy



Class	Name	Example
$\mathcal{O}(1)$	Constant	Array index / Queue pop
$\mathcal{O}(\log N)$	Logarithmic	Binary Search (Week 2)
$\mathcal{O}(N)$	Linear	Two-pointer Merge
$\mathcal{O}(N \log N)$	**Log-linear**	**Merge Sort / Timsort**
$\mathcal{O}(N^2)$	Quadratic	Bubble / Insertion Sort

Memory Layouts: Contiguous vs. Linked Structures

1. Contiguous Memory (Dynamic Array / `list`)

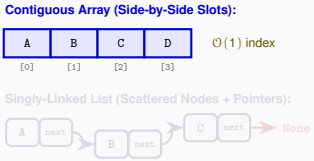
Elements (or pointers) sit side-by-side in an unbroken memory block (Week 2):

- **Pros:** Instant $\mathcal{O}(1)$ random access (`L[i]`); high **spatial locality** (CPU cache friendly).
- **Cons:** Front removal (`pop(0)`) costs $\mathcal{O}(N)$ because $N - 1$ elements must shift.

2. Linked Memory (Linked List)

Elements live in separate heap **Nodes**, each storing a **value** and a **pointer** (`next`) to the following node:

- **Pros:** Inserting/deleting at a known node (head/tail) costs $\mathcal{O}(1)$ by rewiring pointers—zero shifting!
- **Cons:** No $\mathcal{O}(1)$ indexing: reaching item k requires $\mathcal{O}(N)$ sequential **traversal**.



Head-to-Head Summary

Operation	Dynamic Array	Linked List (Head+Tail)
Index i-th item	$\mathcal{O}(1)$	$\mathcal{O}(N)$ (traversal)
Add at Tail (End)	$\mathcal{O}(1)$ (amort.)	$\mathcal{O}(1)$ (exact)
Pop at Head	$\mathcal{O}(N)$ (shifts!)	$\mathcal{O}(1)$ (pointer update)

Memory Layouts: Contiguous vs. Linked Structures

1. Contiguous Memory (Dynamic Array / `list`)

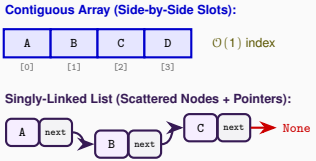
Elements (or pointers) sit side-by-side in an unbroken memory block (Week 2):

- **Pros:** Instant $\mathcal{O}(1)$ random access (`L[i]`); high **spatial locality** (CPU cache friendly).
- **Cons:** Front removal (`pop(0)`) costs $\mathcal{O}(\mathbf{N})$ because $\mathrm{N} - 1$ elements must shift.

2. Linked Memory (Linked List)

Elements live in separate heap **Nodes**, each storing a **value** and a **pointer** (`next`) to the following node:

- **Pros:** Inserting/deleting at a known node (head/tail) costs $\mathcal{O}(\mathbf{1})$ by rewiring pointers—zero shifting!
- **Cons:** No $\mathcal{O}(1)$ indexing: reaching item k requires $\mathcal{O}(\mathbf{N})$ sequential **traversal**.



Head-to-Head Summary

Operation	Dynamic Array	Linked List (Head+Tail)
Index i-th item	$\mathcal{O}(\mathbf{1})$	$\mathcal{O}(\mathrm{N})$ (traversal)
Add at Tail (End)	$\mathcal{O}(1)$ (amort.)	$\mathcal{O}(\mathbf{1})$ (exact)
Pop at Head	$\mathcal{O}(\mathrm{N})$ (shifts!)	$\mathcal{O}(\mathbf{1})$ (pointer update)

Memory Layouts: Contiguous vs. Linked Structures

1. Contiguous Memory (Dynamic Array / `list`)

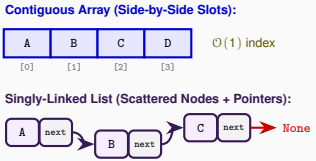
Elements (or pointers) sit side-by-side in an unbroken memory block (Week 2):

- **Pros:** Instant $\mathcal{O}(1)$ random access (`L[i]`); high **spatial locality** (CPU cache friendly).
- **Cons:** Front removal (`pop(0)`) costs $\mathcal{O}(\mathbf{N})$ because $\mathrm{N} - 1$ elements must shift.

2. Linked Memory (Linked List)

Elements live in separate heap **Nodes**, each storing a **value** and a **pointer** (`next`) to the following node:

- **Pros:** Inserting/deleting at a known node (head/tail) costs $\mathcal{O}(\mathbf{1})$ by rewiring pointers—zero shifting!
- **Cons:** No $\mathcal{O}(1)$ indexing: reaching item k requires $\mathcal{O}(\mathbf{N})$ sequential **traversal**.



Head-to-Head Summary

Operation	Dynamic Array	Linked List (Head+Tail)
Index `i`-th item	$\mathcal{O}(\mathbf{1})$	$\mathcal{O}(\mathrm{N})$ (traversal)
Add at Tail (End)	$\mathcal{O}(1)$ (amort.)	$\mathcal{O}(\mathbf{1})$ (exact)
Pop at Head	$\mathcal{O}(\mathrm{N})$ (shifts!)	$\mathcal{O}(\mathbf{1})$ (pointer update)

The Queue ADT and Implementations

The Queue ADT: First-In, First-Out (FIFO)

Definition & FIFO Principle

A **Queue** is a linear ADT where elements are added at the **tail (rear)** and removed from the **head (front)**:

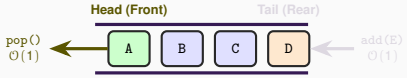
- **FIFO (First-In, First-Out):** Elements are processed in the exact order they arrived, like passengers queueing for a lift.

Core ADT Interface & Complexity Targets

`add(val)` (Enqueue): Insert `val` at the **tail** in $\mathcal{O}(1)$ time.

`pop()` (Dequeue): Remove and return the item at the **head** in $\mathcal{O}(1)$ time.

`is_empty()`: Return `True` if empty in $\mathcal{O}(1)$ time.



Engineering Applications

- **Lift Control (Week 1):** FIFO hall-call queue serving floor requests in arrival order.
- **OS & Networks:** CPU task scheduling, print spools, and packet buffers.
- **Search:** Breadth-First Search (BFS) level-by-level exploration.

The Queue ADT: First-In, First-Out (FIFO)

Definition & FIFO Principle

A **Queue** is a linear ADT where elements are added at the **tail (rear)** and removed from the **head (front)**:

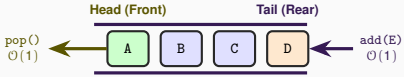
- **FIFO (First-In, First-Out):** Elements are processed in the exact order they arrived, like passengers queueing for a lift.

Core ADT Interface & Complexity Targets

`add(val)` (Enqueue): Insert `val` at the **tail** in $\mathcal{O}(\mathbf{1})$ time.

`pop()` (Dequeue): Remove and return the item at the **head** in $\mathcal{O}(\mathbf{1})$ time.

`is_empty()`: Return `True` if empty in $\mathcal{O}(\mathbf{1})$ time.



Engineering Applications

- **Lift Control (Week 1):** FIFO hall-call queue serving floor requests in arrival order.
- **OS & Networks:** CPU task scheduling, print spools, and packet buffers.
- **Search:** Breadth-First Search (BFS) level-by-level exploration.

The Queue ADT: First-In, First-Out (FIFO)

Definition & FIFO Principle

A **Queue** is a linear ADT where elements are added at the **tail (rear)** and removed from the **head (front)**:

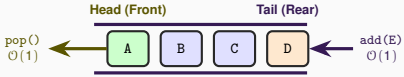
- **FIFO (First-In, First-Out):** Elements are processed in the exact order they arrived, like passengers queueing for a lift.

Core ADT Interface & Complexity Targets

`add(val)` (Enqueue): Insert `val` at the **tail** in $\mathcal{O}(1)$ time.

`pop()` (Dequeue): Remove and return the item at the **head** in $\mathcal{O}(1)$ time.

`is_empty()`: Return `True` if empty in $\mathcal{O}(1)$ time.



Engineering Applications

- **Lift Control (Week 1):** FIFO hall-call queue serving floor requests in arrival order.
- **OS & Networks:** CPU task scheduling, print spools, and packet buffers.
- **Search:** Breadth-First Search (BFS) level-by-level exploration.

Why Not a Python `list`? The $\mathcal{O}(N)$ Dequeue Pitfall

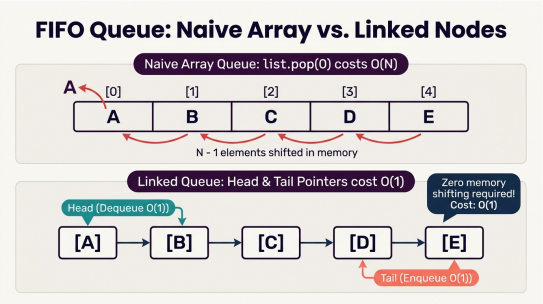
Naive Array-Based Queue

Suppose we use a standard Python `list` as a queue:

```
1 q = []
2 # Enqueue at the end (Tail):
3 q.append("Task A")    # O(1) amortised
4 q.append("Task B")    # O(1) amortised
5
6 # Dequeue from the front (Head):
7 first = q.pop(0)      # O(N) - SLOW!
```

Why `q.pop(0)` Violates Our $\mathcal{O}(1)$ Goal

As we proved in Week 2 (`testdel.py`), removing index 0 from a contiguous array forces CPython to shift all remaining $N - 1$ pointers one slot left ($\mathcal{O}(N)$ per pop, or $\mathcal{O}(N^2)$ to drain N items!).



Why Not a Python `list`? The $\mathcal{O}(N)$ Dequeue Pitfall

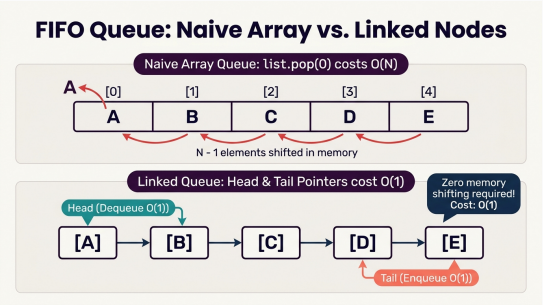
Naive Array-Based Queue

Suppose we use a standard Python `list` as a queue:

```
1 q = []
2 # Enqueue at the end (Tail):
3 q.append("Task A")    # O(1) amortised
4 q.append("Task B")    # O(1) amortised
5
6 # Dequeue from the front (Head):
7 first = q.pop(0)      # O(N) - SLOW!
```

Why `q.pop(0)` Violates Our $\mathcal{O}(1)$ Goal

As we proved in Week 2 (`testdel.py`), removing index 0 from a contiguous array forces CPython to shift all remaining $N - 1$ pointers one slot left ($\mathcal{O}(N)$ per pop, or $\mathcal{O}(N^2)$ to drain N items!).



Building Block: The Singly-Linked `Node` (`simple_node.py`)

Node Implementation (`src/simple_node.py`)

```
1 class Node():
2     def __init__(self, value):
3         self.value, self.next = value, None
4     def append(self, node):
5         if self.next is not None:
6             raise ValueError("next node is not
      none")
7         self.next = node
8     def __eq__(self, other):
9         return self.value == other.value
```

Unit Testing & Traversal (`simple_node.py`)

```
1 def test_node():
2     n1, n2, n3 = Node(1), Node(2), Node(3)
3     n1.append(n2); n2.append(n3)
4     # Sequential traversal: O(N)
5     node, count = n1, 1
6     while node.next is not None:
7         node = node.next
8         count += 1
9     assert node == n3 and count == 3
```


Defensive Design & Dunder Methods

`append()` checks `self.next is None` before linking. `__eq__` (and `__str__`) define `==` and `str()`.

Building Block: The Singly-Linked `Node` (`simple_node.py`)

Node Implementation (`src/simple_node.py`)

```
class Node():
    def __init__(self, value):
        self.value, self.next = value, None
    def append(self, node):
        if self.next is not None:
            raise ValueError("next node is not none")
        self.next = node
    def __eq__(self, other):
        return self.value == other.value
```

Unit Testing & Traversal (`simple_node.py`)

```
def test_node():
    n1, n2, n3 = Node(1), Node(2), Node(3)
    n1.append(n2); n2.append(n3)
    # Sequential traversal: O(N)
    node, count = n1, 1
    while node.next is not None:
        node = node.next
        count += 1
    assert node == n3 and count == 3
```

Defensive Design & Dunder Methods

`append()` checks `self.next is None` before linking. `__eq__` (and `__str__`) define == and `str()`.

Building Block: The Singly-Linked `Node` (`simple_node.py`)

Node Implementation (`src/simple_node.py`)

```
class Node():
    def __init__(self, value):
        self.value, self.next = value, None
    def append(self, node):
        if self.next is not None:
            raise ValueError("next node is not none")
        self.next = node
    def __eq__(self, other):
        return self.value == other.value
```

Unit Testing & Traversal (`simple_node.py`)

```
def test_node():
    n1, n2, n3 = Node(1), Node(2), Node(3)
    n1.append(n2); n2.append(n3)
    # Sequential traversal: O(N)
    node, count = n1, 1
    while node.next is not None:
        node = node.next
        count += 1
    assert node == n3 and count == 3
```

Defensive Design & Dunder Methods

`append()` checks `self.next is None` before linking. `__eq__` (and `__str__`) define == and `str()`.



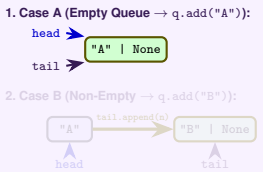
Queue: Enqueue (`custom_queue.py`)

Initialisation & Enqueue

```python
from simple_node import Node

class Queue:
    def __init__(self):
        self.head = None
        self.tail = None
    def is_empty(self):
        return self.head is None
    def add(self, value):
        n = Node(value)
        if self.head is None:
            self.head = n        # Case A: Empty
        else:
            self.tail.append(n) # Case B: Non-empty
        self.tail = n            # Update tail
```

Tracing `q.add(value)` ($\mathcal{O}(1)$ Time)



Why We Need `self.tail`

Without `tail`, `add()` would traverse from `head` ($\mathcal{O}(N)$).
Maintaining `tail` makes `add()` strictly $\mathcal{O}(1)$.

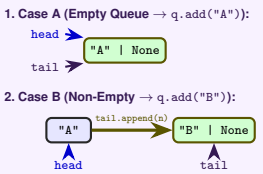
Queue: Enqueue (`custom_queue.py`)

Initialisation & Enqueue

```python
from simple_node import Node

class Queue:
    def __init__(self):
        self.head = None
        self.tail = None
    def is_empty(self):
        return self.head is None
    def add(self, value):
        n = Node(value)
        if self.head is None:
            self.head = n        # Case A: Empty
        else:
            self.tail.append(n) # Case B: Non-empty
        self.tail = n            # Update tail
```

Tracing `q.add(value)` ($\mathcal{O}(1)$ Time)



Why We Need `self.tail`

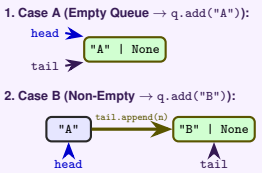
Without `tail`, `add()` would traverse from `head` ($\mathcal{O}(N)$).
Maintaining `tail` makes `add()` strictly $\mathcal{O}(1)$.

Queue: Enqueue (`custom_queue.py`)

Initialisation & Enqueue

```
1  from simple_node import Node
2
3  class Queue:
4      def __init__(self):
5          self.head = None
6          self.tail = None
7      def is_empty(self):
8          return self.head is None
9      def add(self, value):
10         n = Node(value)
11         if self.head is None:
12             self.head = n        # Case A: Empty
13         else:
14             self.tail.append(n) # Case B: Non-
       empty
15         self.tail = n            # Update tail
```

Tracing `q.add(value)` ($\mathcal{O}(1)$ Time)



Why We Need `self.tail`

Without `tail`, `add()` would traverse from `head` ($\mathcal{O}(N)$).
Maintaining `tail` makes `add()` strictly $\mathcal{O}(1)$.

Queue: Dequeue & Invariant (`custom_queue.py`)

Dequeue Implementation (`custom_queue.py`)

```
1     def pop(self):
2         # 1. Fail fast if queue is empty
3         if self.head is None:
4             raise ValueError("Empty queue")
5         # 2. Extract value & advance head
6         value = self.head.value
7         self.head = self.head.next
8         # 3. Edge case: Last item popped!
9         if self.head is None:
10            self.tail = None
11        return value
```

Definition: Class Invariant

A **class invariant** must hold true before and after every method call: `self.head is None` $\iff$ `self.tail is None`.

The "Dangling Tail Pointer" Bug (Step 3)

If we pop the **last node** and forget `self.tail = None`:

- **Memory Leak:** The popped node cannot be garbage-collected because `self.tail` still points to it.
- **Broken Invariant:** `head` is `None` while `tail` points to a removed node!

Queue: Dequeue & Invariant (`custom_queue.py`)

Dequeue Implementation (`custom_queue.py`)

```
1     def pop(self):
2         # 1. Fail fast if queue is empty
3         if self.head is None:
4             raise ValueError("Empty queue")
5         # 2. Extract value & advance head
6         value = self.head.value
7         self.head = self.head.next
8         # 3. Edge case: Last item popped!
9         if self.head is None:
10            self.tail = None
11        return value
```

Definition: Class Invariant

A **class invariant** must hold true before and after every method call: `self.head is None` $\iff$ `self.tail is None`.

The "Dangling Tail Pointer" Bug (Step 3)

If we pop the **last node** and forget `self.tail = None`:

- **Memory Leak:** The popped node cannot be garbage-collected because `self.tail` still points to it.
- **Broken Invariant:** `head` is `None` while `tail` points to a removed node!

Queue: Dequeue & Invariant (custom_queue.py)

Dequeue Implementation (custom_queue.py)

```
1     def pop(self):
2         # 1. Fail fast if queue is empty
3         if self.head is None:
4             raise ValueError("Empty queue")
5         # 2. Extract value & advance head
6         value = self.head.value
7         self.head = self.head.next
8         # 3. Edge case: Last item popped!
9         if self.head is None:
10            self.tail = None
11        return value
```

Definition: Class Invariant

A **class invariant** must hold true before and after every method call: `self.head is None` $\iff$ `self.tail is None`.

The "Dangling Tail Pointer" Bug (Step 3)

If we pop the **last node** and forget `self.tail = None`:



- **Memory Leak:** The popped node cannot be garbage-collected because `self.tail` still points to it.
- **Broken Invariant:** `head` is `None` while `tail` points to a removed node!

Testing Our Queue & Python's `collections.deque`

Unit Testing `Queue` (`src/custom_queue.py`)

```python
def test_queue():
    q = Queue()
    try:
        q.pop()
    except ValueError as err:
        assert "Attempt to pop" in str(err)
    # Verify FIFO order preservation:
    lst = [1, 2, 3, 5, 6, 7, 8, 9, 10]
    for item in lst:
        q.add(item)
    lst2 = []
    while not q.is_empty():
        lst2.append(q.pop())
    assert lst2 == lst
```

Production Best Practice: `collections.deque`

In production Python, use `collections.deque` (**D**ouble-**E**nded **Que**ue, pronounced "deck"):

```python
from collections import deque
q = deque()
q.append("Task 1")      # Enqueue right: O(1)
q.append("Task 2")      # Enqueue right: O(1)
first = q.popleft()     # Dequeue left:  O(1)
```

How CPython Implements `deque` Under the Hood

Instead of allocating one Python node object per item, CPython's `deque` uses a **doubly-linked list of fixed-size 64-pointer C arrays**—combining $\mathcal{O}(1)$ `append()`/`popleft()` with high CPU cache locality!

Testing Our Queue & Python's `collections.deque`

Unit Testing `Queue` (`src/custom_queue.py`)

```python
def test_queue():
    q = Queue()
    try:
        q.pop()
    except ValueError as err:
        assert "Attempt to pop" in str(err)
    # Verify FIFO order preservation:
    lst = [1, 2, 3, 5, 6, 7, 8, 9, 10]
    for item in lst:
        q.add(item)
    lst2 = []
    while not q.is_empty():
        lst2.append(q.pop())
    assert lst2 == lst
```

Production Best Practice: `collections.deque`

In production Python, use `collections.deque` (**D**ouble-**E**nded **Que**ue, pronounced "deck"):

```python
from collections import deque
q = deque()
q.append("Task 1")      # Enqueue right: O(1)
q.append("Task 2")      # Enqueue right: O(1)
first = q.popleft()     # Dequeue left:  O(1)
```

How CPython Implements `deque` Under the Hood

Instead of allocating one Python node object per item, CPython's `deque` uses a **doubly-linked list of fixed-size 64-pointer C arrays**—combining $\mathcal{O}(1)$ `append()`/`popleft()` with high CPU cache locality!

Testing Our Queue & Python's `collections.deque`

Unit Testing `Queue` (`src/custom_queue.py`)

```python
def test_queue():
    q = Queue()
    try:
        q.pop()
    except ValueError as err:
        assert "Attempt to pop" in str(err)
    # Verify FIFO order preservation:
    lst = [1, 2, 3, 5, 6, 7, 8, 9, 10]
    for item in lst:
        q.add(item)
    lst2 = []
    while not q.is_empty():
        lst2.append(q.pop())
    assert lst2 == lst
```

Production Best Practice: `collections.deque`

In production Python, use `collections.deque` (**D**ouble-**E**nded **Que**ue, pronounced "deck"):

```python
from collections import deque
q = deque()
q.append("Task 1")      # Enqueue right: O(1)
q.append("Task 2")      # Enqueue right: O(1)
first = q.popleft()     # Dequeue left:  O(1)
```

How CPython Implements `deque` Under the Hood

Instead of allocating one Python node object per item, CPython's `deque` uses a **doubly-linked list of fixed-size 64-pointer C arrays**—combining $\mathcal{O}(1)$ `append()`/`popleft()` with high CPU cache locality!

The Sorting Problem and Complexity

The Sorting Problem: Formal Specification

Formal Problem Definition

- **Input:** A sequence of N elements $A = (a_1, a_2, \ldots, a_N)$ and a **total order relation** $\leqslant$.
- **Output:** A **permutation** (one-to-one reordering) $A' = (a'_1, a'_2, \ldots, a'_N)$ of A such that $a'_1 \leqslant a'_2 \leqslant \cdots \leqslant a'_N$.

What is a Total Order Relation ($\leqslant$)?

For any elements x, y, z, the comparison $\leqslant$ satisfies:

1. **Reflexivity:** $x \leqslant x$.
2. **Antisymmetry:** If $x \leqslant y$ and $y \leqslant x$, then $x = y$.
3. **Transitivity:** If $x \leqslant y$ and $y \leqslant z$, then $x \leqslant z$.
4. **Totality:** Either $x \leqslant y$ or $y \leqslant x$ (every pair is comparable).

Why is Sorting So Fundamental?

- **Unlocks Fast Search:** Enables $\mathcal{O}(\log N)$ Binary Search (`binarysearch.py`, Week 2) instead of $\mathcal{O}(N)$ linear scan.
- **Data Processing Prerequisite:** Makes duplicate detection, median/quantile finding, and database merge-joins $\mathcal{O}(N)$ operations once sorted.

Problem vs. Algorithm

Sorting is the *problem specification* (what output is required). **Bubble Sort**, **Merge Sort**, **Timsort**, or even **Linear Programming** are distinct *methods* that solve it!

The Sorting Problem: Formal Specification

Formal Problem Definition

- **Input:** A sequence of N elements $A = (a_1, a_2, \ldots, a_N)$ and a **total order relation** $\leqslant$.
- **Output:** A **permutation** (one-to-one reordering) $A' = (a'_1, a'_2, \ldots, a'_N)$ of A such that $a'_1 \leqslant a'_2 \leqslant \cdots \leqslant a'_N$.

What is a Total Order Relation ($\leqslant$)?

For any elements x, y, z, the comparison $\leqslant$ satisfies:

1. **Reflexivity:** $x \leqslant x$.
2. **Antisymmetry:** If $x \leqslant y$ and $y \leqslant x$, then $x = y$.
3. **Transitivity:** If $x \leqslant y$ and $y \leqslant z$, then $x \leqslant z$.
4. **Totality:** Either $x \leqslant y$ or $y \leqslant x$ (every pair is comparable).

Why is Sorting So Fundamental?

- **Unlocks Fast Search:** Enables $\mathcal{O}(\log N)$ Binary Search (`binarysearch.py`, Week 2) instead of $\mathcal{O}(N)$ linear scan.
- **Data Processing Prerequisite:** Makes duplicate detection, median/quantile finding, and database merge-joins $\mathcal{O}(N)$ operations once sorted.

Problem vs. Algorithm

Sorting is the *problem specification* (what output is required). **Bubble Sort**, **Merge Sort**, **Timsort**, or even **Linear Programming** are distinct *methods* that solve it!

The Sorting Problem: Formal Specification

Formal Problem Definition

- **Input:** A sequence of N elements $A = (a_1, a_2, \ldots, a_N)$ and a **total order relation** $\leqslant$.
- **Output:** A **permutation** (one-to-one reordering) $A' = (a'_1, a'_2, \ldots, a'_N)$ of A such that $a'_1 \leqslant a'_2 \leqslant \cdots \leqslant a'_N$.

What is a Total Order Relation ($\leqslant$)?

For any elements x, y, z, the comparison $\leqslant$ satisfies:

1. **Reflexivity:** $x \leqslant x$.
2. **Antisymmetry:** If $x \leqslant y$ and $y \leqslant x$, then $x = y$.
3. **Transitivity:** If $x \leqslant y$ and $y \leqslant z$, then $x \leqslant z$.
4. **Totality:** Either $x \leqslant y$ or $y \leqslant x$ (every pair is comparable).

Why is Sorting So Fundamental?

- **Unlocks Fast Search:** Enables $\mathcal{O}(\log N)$ Binary Search (`binarysearch.py`, Week 2) instead of $\mathcal{O}(N)$ linear scan.
- **Data Processing Prerequisite:** Makes duplicate detection, median/quantile finding, and database merge-joins $\mathcal{O}(N)$ operations once sorted.

Problem vs. Algorithm

Sorting is the *problem specification* (what output is required). **Bubble Sort**, **Merge Sort**, **Timsort**, or even **Linear Programming** are distinct *methods* that solve it!

Sorting as Linear Optimisation (`sorting_linear_program.ipynb`)

Sorting Without Procedural Comparisons

Sorting can be formulated purely as a **Linear Program**:

$$\max_{P \in \mathbb{R}^{n \times n}} \langle w, Px \rangle \quad \text{s.t. } P_{ij} \in [0,1],\ \sum_{i=1}^{n} P_{ij} = 1,\ \sum_{j=1}^{n} P_{ij} = 1$$

Defining Every Mathematical Symbol

- $x \in \mathbb{R}^n$: Input vector of n real numbers; $w = (1, 2, \ldots, n)^\top$: strictly increasing weight vector.
- $\langle u, v \rangle = \sum_{i=1}^{n} u_i v_i$: **Inner (dot) product**. Maximising $\langle w, Px \rangle$ aligns the largest entries of x with the largest weights in w!
- $P \in \mathbb{R}^{n \times n}$: An $n \times n$ **doubly stochastic matrix** (for all i, j, entries are in $[0, 1]$ and rows/columns sum to 1). Its optimal corner is a binary **permutation matrix**.

Concrete 3×3 Permutation Matrix

Sorting $x = [30, 10, 20]^\top$ with $w = [1, 2, 3]^\top$:

$$\underbrace{\begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{bmatrix}}_{P} \underbrace{\begin{bmatrix} 30 \\ 10 \\ 20 \end{bmatrix}}_{x} = \underbrace{\begin{bmatrix} 10 \\ 20 \\ 30 \end{bmatrix}}_{Px}$$

Dot product $\langle w, Px \rangle = 1(10) + 2(20) + 3(30) = \mathbf{140}$ (maximum over all $3! = 6$ permutations).

Takeaway (`sorting_linear_program.ipynb`)

Mirror descent solves this LP in 500 matrix iterations, whereas procedural **Merge Sort** sorts in $\mathcal{O}(N \log N)$ time!

Sorting as Linear Optimisation (`sorting_linear_program.ipynb`)

Sorting Without Procedural Comparisons

Sorting can be formulated purely as a **Linear Program**:

$$\max_{P \in \mathbb{R}^{n \times n}} \langle w, Px \rangle \quad \text{s.t. } P_{ij} \in [0,1],\ \sum_{i=1}^{n} P_{ij} = 1,\ \sum_{j=1}^{n} P_{ij} = 1$$

Defining Every Mathematical Symbol

- $x \in \mathbb{R}^n$: Input vector of n real numbers;
  $w = (1, 2, \ldots, n)^\top$: strictly increasing weight vector.
- $\langle u, v \rangle = \sum_{i=1}^{n} u_i v_i$: **Inner (dot) product**. Maximising $\langle w, Px \rangle$ aligns the largest entries of x with the largest weights in w!
- $P \in \mathbb{R}^{n \times n}$: An $n \times n$ **doubly stochastic matrix** (for all i, j, entries are in $[0, 1]$ and rows/columns sum to 1). Its optimal corner is a binary **permutation matrix**.

Concrete 3×3 Permutation Matrix

Sorting $x = [30, 10, 20]^\top$ with $w = [1, 2, 3]^\top$:

$$\underbrace{\begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{bmatrix}}_{P} \underbrace{\begin{bmatrix} 30 \\ 10 \\ 20 \end{bmatrix}}_{x} = \underbrace{\begin{bmatrix} 10 \\ 20 \\ 30 \end{bmatrix}}_{Px}$$

Dot product $\langle w, Px \rangle = 1(10) + 2(20) + 3(30) = \mathbf{140}$ (maximum over all $3! = 6$ permutations).

Takeaway (`sorting_linear_program.ipynb`)

Mirror descent solves this LP in 500 matrix iterations, whereas procedural **Merge Sort** sorts in $\mathcal{O}(N \log N)$ time!

Sorting as Linear Optimisation (`sorting_linear_program.ipynb`)

Sorting Without Procedural Comparisons

Sorting can be formulated purely as a **Linear Program**:

$$\max_{P \in \mathbb{R}^{n \times n}} \langle w, Px \rangle \quad \text{s.t. } P_{ij} \in [0,1],\ \sum_{i=1}^{n} P_{ij} = 1,\ \sum_{j=1}^{n} P_{ij} = 1$$

Defining Every Mathematical Symbol

- $x \in \mathbb{R}^n$: Input vector of n real numbers;
  $w = (1, 2, \ldots, n)^\top$: strictly increasing weight vector.
- $\langle u, v \rangle = \sum_{i=1}^{n} u_i v_i$: **Inner (dot) product**. Maximising $\langle w, Px \rangle$ aligns the largest entries of x with the largest weights in w!
- $P \in \mathbb{R}^{n \times n}$: An $n \times n$ **doubly stochastic matrix** (for all i, j, entries are in $[0, 1]$ and rows/columns sum to 1). Its optimal corner is a binary **permutation matrix**.

Concrete 3×3 Permutation Matrix

Sorting $x = [30, 10, 20]^\top$ with $w = [1, 2, 3]^\top$:

$$\underbrace{\begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{bmatrix}}_{P} \underbrace{\begin{bmatrix} 30 \\ 10 \\ 20 \end{bmatrix}}_{x} = \underbrace{\begin{bmatrix} 10 \\ 20 \\ 30 \end{bmatrix}}_{Px}$$

Dot product $\langle w, Px \rangle = 1(10) + 2(20) + 3(30) = \mathbf{140}$ (maximum over all $3! = 6$ permutations).

Takeaway (`sorting_linear_program.ipynb`)

Mirror descent solves this LP in 500 matrix iterations, whereas procedural **Merge Sort** sorts in $\mathcal{O}(N \log N)$ time!

Classifying Sorting Algorithms: Key Properties

Three Essential Properties Beyond Time

1. **In-Place ($\mathcal{O}(1)$ Extra Space):** Sorts elements inside the original array using only a constant number of extra variables, rather than allocating an $\mathcal{O}(N)$ copy.
2. **Stability:** If two items have **equal keys**, they remain in the **exact same relative order** in the output as in the input.
3. **Adaptivity:** Runs faster (e.g. $\mathcal{O}(N)$ instead of $\mathcal{O}(N \log N)$) when the input is already or nearly sorted.

Why Stability Matters in Engineering

Suppose records are sorted alphabetically by name, and we now sort by **score**:



A **stable** sort keeps `(Alice, 10)` before `(Charlie, 10)`!

Algorithm	Worst	In-Place	Stable	Adaptive
Insertion Sort	$\mathcal{O}(N^2)$	Yes	Yes	Yes ($\mathcal{O}(N)$)
Quick Sort	$\mathcal{O}(N^2)$	Yes	No	No
Merge Sort	$\mathcal{O}(N \log N)$	**No**	**Yes**	No
Timsort	$\mathcal{O}(N \log N)$	No	**Yes**	**Yes ($\mathcal{O}(N)$)**

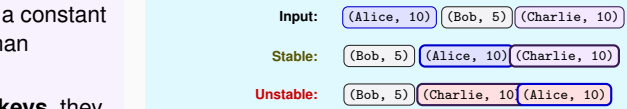
Classifying Sorting Algorithms: Key Properties

Three Essential Properties Beyond Time

1. **In-Place ($\mathcal{O}(1)$ Extra Space):** Sorts elements inside the original array using only a constant number of extra variables, rather than allocating an $\mathcal{O}(N)$ copy.
2. **Stability:** If two items have **equal keys**, they remain in the **exact same relative order** in the output as in the input.
3. **Adaptivity:** Runs faster (e.g. $\mathcal{O}(N)$ instead of $\mathcal{O}(N \log N)$) when the input is already or nearly sorted.

Why Stability Matters in Engineering

Suppose records are sorted alphabetically by name, and we now sort by **score**:



A **stable** sort keeps `(Alice, 10)` before `(Charlie, 10)`!

Algorithm	Worst	In-Place	Stable	Adaptive
Insertion Sort	$\mathcal{O}(N^2)$	Yes	Yes	Yes ($\mathcal{O}(N)$)
Quick Sort	$\mathcal{O}(N^2)$	Yes	No	No
Merge Sort	$\mathcal{O}(N \log N)$	**No**	**Yes**	No
Timsort	$\mathcal{O}(N \log N)$	No	**Yes**	**Yes ($\mathcal{O}(N)$)**

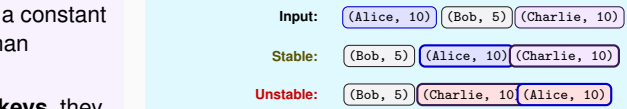
Classifying Sorting Algorithms: Key Properties

Three Essential Properties Beyond Time

1. **In-Place ($\mathcal{O}(1)$ Extra Space):** Sorts elements inside the original array using only a constant number of extra variables, rather than allocating an $\mathcal{O}(N)$ copy.
2. **Stability:** If two items have **equal keys**, they remain in the **exact same relative order** in the output as in the input.
3. **Adaptivity:** Runs faster (e.g. $\mathcal{O}(N)$ instead of $\mathcal{O}(N \log N)$) when the input is already or nearly sorted.

Why Stability Matters in Engineering

Suppose records are sorted alphabetically by name, and we now sort by **score**:



A **stable** sort keeps `(Alice, 10)` before `(Charlie, 10)`!

Algorithm	Worst	In-Place	Stable	Adaptive
Insertion Sort	$\mathcal{O}(N^2)$	Yes	Yes	Yes ($\mathcal{O}(N)$)
Quick Sort	$\mathcal{O}(N^2)$	Yes	No	No
Merge Sort	$\mathcal{O}(N \log N)$	**No**	**Yes**	No
Timsort	$\mathcal{O}(N \log N)$	No	**Yes**	**Yes ($\mathcal{O}(N)$)**

The Comparison Model and the $\Omega(N \log N)$ Lower Bound

Theorem: Comparison Sorting Lower Bound

Any sorting algorithm that determines order solely via pairwise comparisons ($a_i \leqslant a_j$) requires **at least** $\Omega(N \log N)$ **comparisons** in the worst case.

Decision Tree Proof

1. **Permutations ($N!$):** N items have $N! = 1 \times 2 \times \cdots \times N$ possible orderings, each requiring a distinct leaf in the binary decision tree.
2. **Tree Height (H):** Each Yes/No comparison branches in 2, so a tree of height H has at most 2^H leaves.
3. **Logarithmic Bound ($\log_2$):** To cover all $N!$ leaves, $2^H \geqslant N! \implies H \geqslant \log_2(N!)$.
4. **Stirling's Approximation:** Since $N/2$ factors in $N!$ are $\geqslant N/2$, $\log_2(N!) \geqslant \frac{N}{2} \log_2(N/2) = \Omega(N \log N)$.

Decision Tree for $N = 3$ ($3! = 6$ Leaves)

For $N = 3$, there are $3! = 6$ leaves, so $2^H \geqslant 6 \implies H \geqslant 3$ comparisons in the worst case.

The Comparison Model and the $\Omega(N \log N)$ Lower Bound

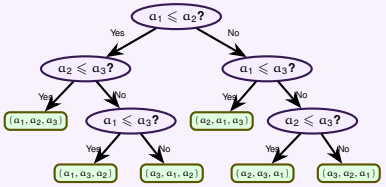
Theorem: Comparison Sorting Lower Bound

Any sorting algorithm that determines order solely via pairwise comparisons ($a_i \leqslant a_j$) requires **at least** $\Omega(N \log N)$ **comparisons** in the worst case.

Decision Tree Proof

1. **Permutations ($N!$):** N items have $N! = 1 \times 2 \times \cdots \times N$ possible orderings, each requiring a distinct leaf in the binary decision tree.
2. **Tree Height (H):** Each Yes/No comparison branches in 2, so a tree of height H has at most 2^H leaves.
3. **Logarithmic Bound ($\log_2$):** To cover all $N!$ leaves, $2^H \geqslant N! \implies H \geqslant \log_2(N!)$.
4. **Stirling's Approximation:** Since $N/2$ factors in $N!$ are $\geqslant N/2$, $\log_2(N!) \geqslant \frac{N}{2} \log_2(N/2) = \Omega(N \log N)$.

Decision Tree for $N = 3$ ($3! = 6$ Leaves)



For $N = 3$, there are $3! = 6$ leaves, so $2^H \geqslant 6 \implies H \geqslant 3$ comparisons in the worst case.

The Comparison Model and the $\Omega(N \log N)$ Lower Bound

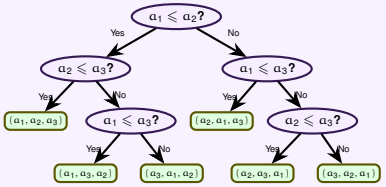
Theorem: Comparison Sorting Lower Bound

Any sorting algorithm that determines order solely via pairwise comparisons ($a_i \leqslant a_j$) requires **at least** $\Omega(N \log N)$ **comparisons** in the worst case.

Decision Tree Proof

1. **Permutations ($N!$):** N items have $N! = 1 \times 2 \times \cdots \times N$ possible orderings, each requiring a distinct leaf in the binary decision tree.
2. **Tree Height (H):** Each Yes/No comparison branches in 2, so a tree of height H has at most 2^H leaves.
3. **Logarithmic Bound ($\log_2$):** To cover all $N!$ leaves, $2^H \geqslant N! \implies H \geqslant \log_2(N!)$.
4. **Stirling's Approximation:** Since $N/2$ factors in $N!$ are $\geqslant N/2$, $\log_2(N!) \geqslant \frac{N}{2} \log_2(N/2) = \Omega(N \log N)$.

Decision Tree for $N = 3$ ($3! = 6$ Leaves)



For $N = 3$, there are $3! = 6$ leaves, so $2^H \geqslant 6 \implies H \geqslant 3$ comparisons in the worst case.

The Comparison Model and the $\Omega(N \log N)$ Lower Bound

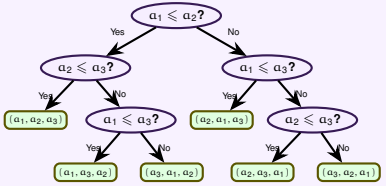
Theorem: Comparison Sorting Lower Bound

Any sorting algorithm that determines order solely via pairwise comparisons ($a_i \leqslant a_j$) requires **at least** $\Omega(N \log N)$ **comparisons** in the worst case.

Decision Tree Proof

1. **Permutations ($N!$):** N items have $N! = 1 \times 2 \times \cdots \times N$ possible orderings, each requiring a distinct leaf in the binary decision tree.
2. **Tree Height (H):** Each Yes/No comparison branches in 2, so a tree of height H has at most 2^H leaves.
3. **Logarithmic Bound ($\log_2$):** To cover all $N!$ leaves, $2^H \geqslant N! \implies H \geqslant \log_2(N!)$.
4. **Stirling's Approximation:** Since $N/2$ factors in $N!$ are $\geqslant N/2$, $\log_2(N!) \geqslant \frac{N}{2} \log_2(N/2) = \boldsymbol{\Omega(N \log N)}$.

Decision Tree for $N = 3$ ($3! = 6$ Leaves)



For $N = 3$, there are $3! = 6$ leaves, so $2^H \geqslant 6 \implies H \geqslant 3$ comparisons in the worst case.

Merge Sort: Divide and Conquer

Divide and Conquer: Merge Sort (`merge.py`)

The Divide & Conquer Paradigm

Invented by John von Neumann (1945), **Merge Sort** achieves the optimal $\mathcal{O}(N \log N)$ bound in three steps:

1. **Divide:** Split the list at midpoint `pivot = len(lst) // 2` into two halves of size $\approx N/2$.
2. **Conquer:** Sort both halves by calling `mergesort` on each half until reaching the **base case** (`len(lst) <= 1`).
3. **Combine:** Merge the two sorted halves into one sorted list in $\mathcal{O}(N)$ linear time.

Terminology: Recursion & Base Case

A **recursive function** calls itself on smaller inputs until hitting a non-recursive **base case** (*explored in depth in Week 5*).

Implementation (`src/merge.py`)

```python
def mergesort(lst, compare):
    if len(lst) <= 1:  # Base case
        return lst
    pivot = len(lst) // 2
    lst1 = mergesort(lst[:pivot], compare)
    lst2 = mergesort(lst[pivot:], compare)
    return merge(lst1, lst2, compare)
```

Higher-Order Design

`mergesort` accepts a **comparison function** `compare(a, b)` so it can sort by any custom ordering rule!

Divide and Conquer: Merge Sort (`merge.py`)

The Divide & Conquer Paradigm

Invented by John von Neumann (1945), **Merge Sort** achieves the optimal $\mathcal{O}(N \log N)$ bound in three steps:

1. **Divide:** Split the list at midpoint `pivot = len(lst) // 2` into two halves of size $\approx N/2$.
2. **Conquer:** Sort both halves by calling `mergesort` on each half until reaching the **base case** (`len(lst) <= 1`).
3. **Combine:** Merge the two sorted halves into one sorted list in $\mathcal{O}(N)$ linear time.

Terminology: Recursion & Base Case

A **recursive function** calls itself on smaller inputs until hitting a non-recursive **base case** (*explored in depth in Week 5*).

Implementation (`src/merge.py`)

```python
def mergesort(lst, compare):
    if len(lst) <= 1:  # Base case
        return lst
    pivot = len(lst) // 2
    lst1 = mergesort(lst[:pivot], compare)
    lst2 = mergesort(lst[pivot:], compare)
    return merge(lst1, lst2, compare)
```

Higher-Order Design

`mergesort` accepts a **comparison function** `compare(a, b)` so it can sort by any custom ordering rule!

Divide and Conquer: Merge Sort (`merge.py`)

The Divide & Conquer Paradigm

Invented by John von Neumann (1945), **Merge Sort** achieves the optimal $\mathcal{O}(N \log N)$ bound in three steps:

1. **Divide:** Split the list at midpoint `pivot = len(lst) // 2` into two halves of size $\approx N/2$.
2. **Conquer:** Sort both halves by calling `mergesort` on each half until reaching the **base case** (`len(lst) <= 1`).
3. **Combine:** Merge the two sorted halves into one sorted list in $\mathcal{O}(N)$ linear time.

Terminology: Recursion & Base Case

A **recursive function** calls itself on smaller inputs until hitting a non-recursive **base case** (*explored in depth in Week 5*).

Implementation (`src/merge.py`)

```python
def mergesort(lst, compare):
    if len(lst) <= 1:  # Base case
        return lst
    pivot = len(lst) // 2
    lst1 = mergesort(lst[:pivot], compare)
    lst2 = mergesort(lst[pivot:], compare)
    return merge(lst1, lst2, compare)
```

Higher-Order Design

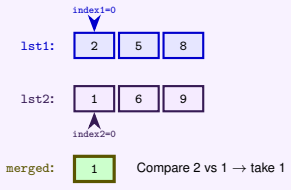
`mergesort` accepts a **comparison function** `compare(a, b)` so it can sort by any custom ordering rule!

The Core Operation: Linear-Time Two-Pointer Merge (`merge.py`)

Two-Pointer Merge (`src/merge.py`)

```
1  def merge(lst1, lst2, compare):
2      merged_list = []
3      index1, index2 = 0, 0
4      len1, len2 = len(lst1), len(lst2)
5      while index1 < len1 or index2 < len2:
6          if index2 == len2 or (index1 < len1 and
7             compare(lst1[index1], lst2[index2])):
8              merged_list.append(lst1[index1])
9              index1 += 1
10         else:
11             merged_list.append(lst2[index2])
12             index2 += 1
13     return merged_list
```

Visual Trace: Merging Two Sorted Lists



Why `merge()` is Strictly $\mathcal{O}(N)$

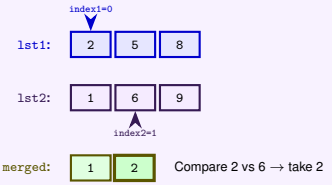
Each step appends **1 item** and advances `index1` or `index2`: exactly $|L_1| + |L_2| = N$ iterations $\implies \mathcal{O}(N)$.

The Core Operation: Linear-Time Two-Pointer Merge (`merge.py`)

Two-Pointer Merge (`src/merge.py`)

```python
def merge(lst1, lst2, compare):
    merged_list = []
    index1, index2 = 0, 0
    len1, len2 = len(lst1), len(lst2)
    while index1 < len1 or index2 < len2:
        if index2 == len2 or (index1 < len1 and
           compare(lst1[index1], lst2[index2])):
            merged_list.append(lst1[index1])
            index1 += 1
        else:
            merged_list.append(lst2[index2])
            index2 += 1
    return merged_list
```

Visual Trace: Merging Two Sorted Lists



Why `merge()` is Strictly $\mathcal{O}(N)$

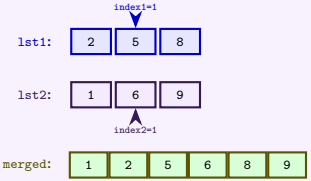
Each step appends **1 item** and advances `index1` or `index2`: exactly $|L_1| + |L_2| = N$ iterations $\implies \mathcal{O}(N)$.

The Core Operation: Linear-Time Two-Pointer Merge (`merge.py`)

Two-Pointer Merge (`src/merge.py`)

```python
def merge(lst1, lst2, compare):
    merged_list = []
    index1, index2 = 0, 0
    len1, len2 = len(lst1), len(lst2)
    while index1 < len1 or index2 < len2:
        if index2 == len2 or (index1 < len1 and
           compare(lst1[index1], lst2[index2])):
            merged_list.append(lst1[index1])
            index1 += 1
        else:
            merged_list.append(lst2[index2])
            index2 += 1
    return merged_list
```

Visual Trace: Merging Two Sorted Lists



Why `merge()` is Strictly $\mathcal{O}(N)$

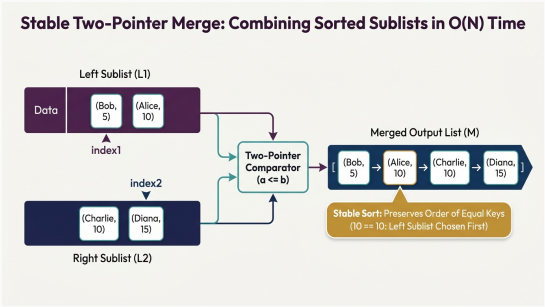
Each step appends **1 item** and advances `index1` or `index2`: exactly $|L_1| + |L_2| = N$ iterations $\implies \mathcal{O}(N)$.

Critical Analysis: Is `merge.py` Stable?

Inspecting the Tie-Breaking Condition

Look at `src/merge.py` when `lst1[index1]` (left half) and `lst2[index2]` (right half) have **equal keys**:

```
1 if index2 == len2 or (index1 < len1 and
2    compare(lst1[index1], lst2[index2])):
3     merged_list.append(lst1[index1]) # Left
4     index1 += 1
5 else:
6     merged_list.append(lst2[index2]) # Right
7     index2 += 1
```



Strict < Breaks Stability; <= Restores It!

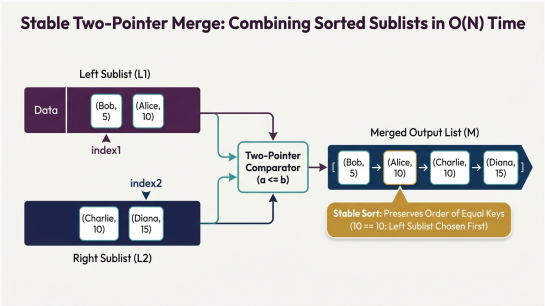
- With `a < b`: When `a == b`, `compare` returns `False`, so `else` picks `lst2` (right half) first—**unstable**!
- Fix (`a <= b`): Returns `True` on ties, picking `lst1` (left half) first and preserving **stability**.

Critical Analysis: Is `merge.py` Stable?

Inspecting the Tie-Breaking Condition

Look at `src/merge.py` when `lst1[index1]` (left half) and `lst2[index2]` (right half) have **equal keys**:

```
1 if index2 == len2 or (index1 < len1 and
2    compare(lst1[index1], lst2[index2])):
3     merged_list.append(lst1[index1]) # Left
4     index1 += 1
5 else:
6     merged_list.append(lst2[index2]) # Right
7     index2 += 1
```



Strict < Breaks Stability; <= Restores It!

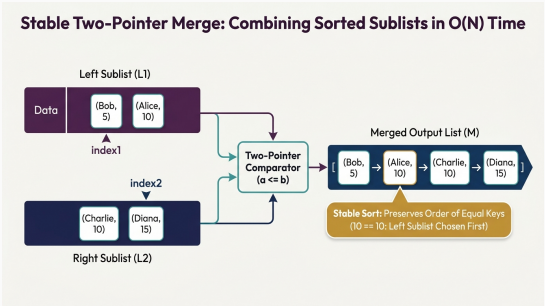
- **With `a < b`:** When $a == b$, `compare` returns `False`, so `else` picks `lst2` (right half) first—**unstable**!
- **Fix (`a <= b`):** Returns `True` on ties, picking `lst1` (left half) first and preserving **stability**.

Critical Analysis: Is `merge.py` Stable?

Inspecting the Tie-Breaking Condition

Look at `src/merge.py` when `lst1[index1]` (left half) and `lst2[index2]` (right half) have **equal keys**:

```
1 if index2 == len2 or (index1 < len1 and
2    compare(lst1[index1], lst2[index2])):
3     merged_list.append(lst1[index1]) # Left
4     index1 += 1
5 else:
6     merged_list.append(lst2[index2]) # Right
7     index2 += 1
```



Strict < Breaks Stability; <= Restores It!

- **With `a < b`:** When $a == b$, `compare` returns `False`, so `else` picks `lst2` (right half) first—**unstable**!
- **Fix (`a <= b`):** Returns `True` on ties, picking `lst1` (left half) first and preserving **stability**.

Merge Sort Complexity: Recurrence Relation & Recursion Tree

What is a Recurrence Relation?

A **recurrence relation** expresses runtime $T(N)$ in terms of smaller subproblems:

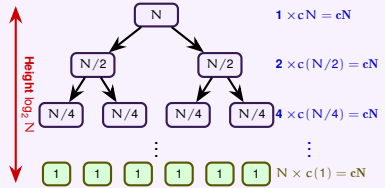
- **Base Case ($N = 1$):** $T(1) = c$ ($\mathcal{O}(1)$).
- **Recursive Step ($N > 1$):**

$$T(N) = \underbrace{2\,T(N/2)}_{\text{2 halves}} + \underbrace{cN}_{\text{merge + slice}}$$

Total Work Across All Levels

- Depth k has 2^k subproblems of size $N/2^k$.
- Work per level $= 2^k \cdot c(N/2^k) = cN$.
- Halving N reaches 1 in $\log_2 N$ splits ($\log_2 N + 1$ levels).
- Total Time $= cN(\log_2 N + 1) = \mathcal{O}(N \log N)$.

Visualising the Recursion Tree



Auxiliary Space: $\mathcal{O}(N)$ (merged lists). Since $\mathcal{O}(N \log N)$ matches the $\Omega(N \log N)$ lower bound, Merge Sort is **asymptotically optimal**!

Merge Sort Complexity: Recurrence Relation & Recursion Tree

What is a Recurrence Relation?

A **recurrence relation** expresses runtime $T(N)$ in terms of smaller subproblems:

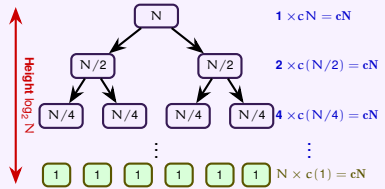
- **Base Case ($N = 1$):** $T(1) = c$ ($\mathcal{O}(1)$).
- **Recursive Step ($N > 1$):**

$$T(N) = \underbrace{2\,T(N/2)}_{\text{2 halves}} + \underbrace{c\,N}_{\text{merge + slice}}$$

Total Work Across All Levels

- Depth k has 2^k subproblems of size $N/2^k$.
- Work per level $= 2^k \cdot c(N/2^k) = cN$.
- Halving N reaches 1 in $\log_2 N$ splits ($\log_2 N + 1$ levels).
- Total Time $= cN(\log_2 N + 1) = \mathcal{O}(N \log N)$.

Visualising the Recursion Tree



Auxiliary Space: $\mathcal{O}(N)$ (merged lists). Since $\mathcal{O}(N \log N)$ matches the $\Omega(N \log N)$ lower bound, Merge Sort is **asymptotically optimal**!

Generalisation, Higher-Order Functions, and Timsort

Higher-Order Functions and Custom Comparators (`merge.py`)

First-Class & Higher-Order Functions

- **First-Class Functions:** In Python, functions are objects—they can be stored in variables and passed as arguments.
- **Higher-Order Function (HOF):** A function that takes a function as an argument (like `mergesort(lst, compare)`) or returns a function.
- **Anonymous Function (`lambda`):** A concise one-line function defined inline without `def`: `lambda x, y: len(x) < len(y)`.

Custom Comparators in `src/merge.py`

```python
# 1. Named comparator: case-insensitive sort
def cmp_str(s1, s2):
    return s1.lower() < s2.lower()

def test_sortstr():
    assert mergesort(["a"], cmp_str) == ["a"]
    assert mergesort(
        ["BA", "aa", "bbbb", "a"], cmp_str
    ) == ["a", "aa", "BA", "bbbb"]
    # 2. Inline lambda: sort by string length!
    assert mergesort(
        ["BAAAAA", "aa", "bbbb", "a"],
        lambda x, y: len(x) < len(y)
    ) == ["a", "aa", "bbbb", "BAAAAA"]
```

Higher-Order Functions and Custom Comparators (`merge.py`)

First-Class & Higher-Order Functions

- **First-Class Functions:** In Python, functions are objects—they can be stored in variables and passed as arguments.
- **Higher-Order Function (HOF):** A function that takes a function as an argument (like `mergesort(lst, compare)`) or returns a function.
- **Anonymous Function (`lambda`):** A concise one-line function defined inline without `def`: `lambda x, y: len(x) < len(y)`.

Custom Comparators in `src/merge.py`

```python
# 1. Named comparator: case-insensitive sort
def cmp_str(s1, s2):
    return s1.lower() < s2.lower()

def test_sortstr():
    assert mergesort(["a"], cmp_str) == ["a"]
    assert mergesort(
        ["BA", "aa", "bbbb", "a"], cmp_str
    ) == ["a", "aa", "BA", "bbbb"]
    # 2. Inline lambda: sort by string length!
    assert mergesort(
        ["BAAAAA", "aa", "bbbb", "a"],
        lambda x, y: len(x) < len(y)
    ) == ["a", "aa", "bbbb", "BAAAAA"]
```



Python's Idiomatic `key=` Parameter vs. Comparators

Why Python Prefers `key=` Over `compare(a, b)`

- **Comparator (`compare(a, b)`):** Takes *two* items and runs on **every comparison** ($\mathcal{O}(\mathbf{N} \log \mathbf{N})$ Python calls!).
- **Key Function (`key=f`):** Takes *one* item and extracts a sortable key **once per item** ($\mathcal{O}(\mathbf{N})$ calls!).

```
1 data = ["BAAAAA", "aa", "bbbb", "a"]
2 # Extract key once per element (O(N) calls):
3 by_len  = sorted(data, key=len)
4 by_case = sorted(data, key=str.lower)
5
6 # Sort tuples by 2nd field with a lambda key:
7 records = [("Alice", 10), ("Bob", 5)]
8 by_score = sorted(records, key=lambda r: r
      [1])
```

Decorate–Sort–Undecorate Pipeline

Because step 3 compares fast C keys without calling Python on every comparison, `sorted(..., key=...)` is orders of magnitude faster!

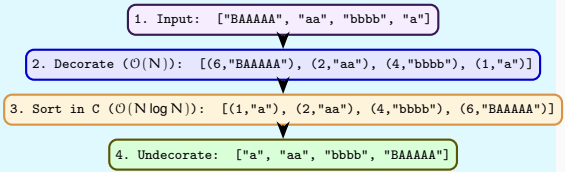
Python's Idiomatic `key=` Parameter vs. Comparators

Why Python Prefers `key=` Over `compare(a, b)`

- **Comparator (`compare(a, b)`):** Takes *two* items and runs on **every comparison** ($\mathcal{O}(\mathbf{N} \log \mathbf{N})$ Python calls!).
- **Key Function (`key=f`):** Takes *one* item and extracts a sortable key **once per item** ($\mathcal{O}(\mathbf{N})$ calls!).

```
1 data = ["BAAAAA", "aa", "bbbb", "a"]
2 # Extract key once per element (O(N) calls):
3 by_len  = sorted(data, key=len)
4 by_case = sorted(data, key=str.lower)
5
6 # Sort tuples by 2nd field with a lambda key:
7 records = [("Alice", 10), ("Bob", 5)]
8 by_score = sorted(records, key=lambda r: r
      [1])
```

Decorate–Sort–Undecorate Pipeline



Because step 3 compares fast C keys without calling Python on every comparison, `sorted(..., key=...)` is orders of magnitude faster!

Sorting in Production: CPython's Timsort

What Powers `list.sort()` and `sorted()`?

Designed by Tim Peters in 2002 for CPython (and now used in Java, Android, and V8), **Timsort** is a **hybrid, stable, adaptive** sorting algorithm combining **Merge Sort** and **Insertion Sort**.

How Timsort Exploits Real-World Data

Real-world datasets almost always contain pre-existing ordered segments called **natural runs**:

1. **Scan for Runs:** Identifies ascending (or strictly descending, which it reverses in place) runs.
2. **Boost Short Runs:** Uses binary Insertion Sort to extend short runs to 32–64 items.
3. **Smart Merge:** Merges runs on a stack using "galloping" binary search.

Visualising Timsort on Partially Sorted Data

Scenario	Pure Merge Sort	CPython Timsort
Already sorted input	$\mathcal{O}(N \log N)$	$\mathcal{O}(N)$ (1 run!)
Worst-case input	$\mathcal{O}(N \log N)$	$\mathcal{O}(N \log N)$
Stable?	Yes	**Yes**

Sorting in Production: CPython's Timsort

What Powers `list.sort()` and `sorted()`?

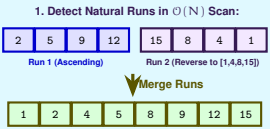
Designed by Tim Peters in 2002 for CPython (and now used in Java, Android, and V8), **Timsort** is a **hybrid, stable, adaptive** sorting algorithm combining **Merge Sort** and **Insertion Sort**.

How Timsort Exploits Real-World Data

Real-world datasets almost always contain pre-existing ordered segments called **natural runs**:

1. **Scan for Runs:** Identifies ascending (or strictly descending, which it reverses in place) runs.
2. **Boost Short Runs:** Uses binary Insertion Sort to extend short runs to 32–64 items.
3. **Smart Merge:** Merges runs on a stack using "galloping" binary search.

Visualising Timsort on Partially Sorted Data



Scenario	Pure Merge Sort	CPython Timsort
Already sorted input	$\mathcal{O}(N \log N)$	$\mathcal{O}(N)$ (1 run!)
Worst-case input	$\mathcal{O}(N \log N)$	$\mathcal{O}(N \log N)$
Stable?	Yes	**Yes**

Summary

Lecture Summary

1. ADTs, Memory & Queues

- **ADT vs. Data Structure:** An ADT defines the *interface* (what); a data structure defines the *memory layout* (how).
- **Contiguous vs. Linked:** Arrays give $\mathcal{O}(1)$ indexing but $\mathcal{O}(N)$ front removal (`pop(0)`); linked lists give $\mathcal{O}(1)$ pointer updates at known ends but $\mathcal{O}(N)$ traversal.
- **Queue (FIFO):** Maintaining both `head` and `tail` pointers in a linked list (`custom_queue.py`) achieves $\mathcal{O}(1)$ `add()` and `pop()`. Always preserve the empty-queue invariant (`head is None` $\iff$ `tail is None`).
- **In Production:** Use `collections.deque` for $\mathcal{O}(1)$ `append()` and `popleft()`.

2. Sorting, Merge Sort & Generalisation

- **Lower Bound:** Comparison sorting requires $\Omega(N \log N)$ worst-case comparisons ($2^H \geqslant N!$).
- **Merge Sort** (`merge.py`)**:** Divide-and-conquer solving $T(N) = 2T(N/2) + \mathcal{O}(N) = \mathcal{O}(N \log N)$ time with $\mathcal{O}(N)$ auxiliary space; requires $\leqslant$ in `merge()` for stability.
- **Keys & Timsort:** Prefer `key=` ($\mathcal{O}(N)$ key calls) over comparators ($\mathcal{O}(N \log N)$ calls). CPython's **Timsort** runs in $\mathcal{O}(N)$ best-case and $\mathcal{O}(N \log N)$ worst-case time.

Coming Up in Week 4

Hierarchical structures (**Binary Search Trees**), lazy evaluation with **Python Generators** (`yield`), and building a **Hash Table** from scratch!

Lecture Summary

1. ADTs, Memory & Queues

- **ADT vs. Data Structure:** An ADT defines the *interface* (what); a data structure defines the *memory layout* (how).
- **Contiguous vs. Linked:** Arrays give $\mathcal{O}(1)$ indexing but $\mathcal{O}(N)$ front removal (`pop(0)`); linked lists give $\mathcal{O}(1)$ pointer updates at known ends but $\mathcal{O}(N)$ traversal.
- **Queue (FIFO):** Maintaining both `head` and `tail` pointers in a linked list (`custom_queue.py`) achieves $\mathcal{O}(1)$ `add()` and `pop()`. Always preserve the empty-queue invariant (`head is None` $\iff$ `tail is None`).
- **In Production:** Use `collections.deque` for $\mathcal{O}(1)$ `append()` and `popleft()`.

2. Sorting, Merge Sort & Generalisation

- **Lower Bound:** Comparison sorting requires $\Omega(N \log N)$ worst-case comparisons ($2^H \geqslant N!$).
- **Merge Sort** (`merge.py`)**:** Divide-and-conquer solving $T(N) = 2T(N/2) + \mathcal{O}(N) = \mathcal{O}(N \log N)$ time with $\mathcal{O}(N)$ auxiliary space; requires $\leqslant$ in `merge()` for stability.
- **Keys & Timsort:** Prefer `key=` ($\mathcal{O}(N)$ key calls) over comparators ($\mathcal{O}(N \log N)$ calls). CPython's **Timsort** runs in $\mathcal{O}(N)$ best-case and $\mathcal{O}(N \log N)$ worst-case time.

Coming Up in Week 4

Hierarchical structures (**Binary Search Trees**), lazy evaluation with **Python Generators** (`yield`), and building a **Hash Table** from scratch!

Lecture Summary

1. ADTs, Memory & Queues

- **ADT vs. Data Structure:** An ADT defines the *interface* (what); a data structure defines the *memory layout* (how).
- **Contiguous vs. Linked:** Arrays give $\mathcal{O}(1)$ indexing but $\mathcal{O}(N)$ front removal (`pop(0)`); linked lists give $\mathcal{O}(1)$ pointer updates at known ends but $\mathcal{O}(N)$ traversal.
- **Queue (FIFO):** Maintaining both `head` and `tail` pointers in a linked list (`custom_queue.py`) achieves $\mathcal{O}(1)$ `add()` and `pop()`. Always preserve the empty-queue invariant (`head is None` $\iff$ `tail is None`).
- **In Production:** Use `collections.deque` for $\mathcal{O}(1)$ `append()` and `popleft()`.

2. Sorting, Merge Sort & Generalisation

- **Lower Bound:** Comparison sorting requires $\boldsymbol{\Omega}(\mathbf{N}\log\mathbf{N})$ worst-case comparisons ($2^{\mathrm{H}} \geqslant \mathrm{N}!$).
- **Merge Sort (`merge.py`):** Divide-and-conquer solving $T(N) = 2T(N/2) + \mathcal{O}(N) = \mathcal{O}(\mathbf{N}\log\mathbf{N})$ time with $\mathcal{O}(N)$ auxiliary space; requires $\leqslant$ in `merge()` for **stability**.
- **Keys & Timsort:** Prefer `key=` ($\mathcal{O}(N)$ key calls) over comparators ($\mathcal{O}(N \log N)$ calls). CPython's **Timsort** runs in $\mathcal{O}(N)$ best-case and $\mathcal{O}(N \log N)$ worst-case time.

Coming Up in Week 4

Hierarchical structures (**Binary Search Trees**), lazy evaluation with **Python Generators** (`yield`), and building a **Hash Table** from scratch!

Lecture Summary

1. ADTs, Memory & Queues

- **ADT vs. Data Structure:** An ADT defines the *interface* (what); a data structure defines the *memory layout* (how).
- **Contiguous vs. Linked:** Arrays give $\mathcal{O}(1)$ indexing but $\mathcal{O}(N)$ front removal (`pop(0)`); linked lists give $\mathcal{O}(1)$ pointer updates at known ends but $\mathcal{O}(N)$ traversal.
- **Queue (FIFO):** Maintaining both `head` and `tail` pointers in a linked list (`custom_queue.py`) achieves $\mathcal{O}(1)$ `add()` and `pop()`. Always preserve the empty-queue invariant (`head is None` $\iff$ `tail is None`).
- **In Production:** Use `collections.deque` for $\mathcal{O}(1)$ `append()` and `popleft()`.

2. Sorting, Merge Sort & Generalisation

- **Lower Bound:** Comparison sorting requires $\boldsymbol{\Omega}(\mathbf{N}\log\mathbf{N})$ worst-case comparisons ($2^H \geqslant N!$).
- **Merge Sort (`merge.py`):** Divide-and-conquer solving $T(N) = 2T(N/2) + \mathcal{O}(N) = \mathcal{O}(\mathbf{N}\log\mathbf{N})$ time with $\mathcal{O}(N)$ auxiliary space; requires $\leqslant$ in `merge()` for **stability**.
- **Keys & Timsort:** Prefer `key=` ($\mathcal{O}(N)$ key calls) over comparators ($\mathcal{O}(N \log N)$ calls). CPython's **Timsort** runs in $\mathcal{O}(N)$ best-case and $\mathcal{O}(N \log N)$ worst-case time.

Coming Up in Week 4

Hierarchical structures (**Binary Search Trees**), lazy evaluation with **Python Generators (`yield`)**, and building a **Hash Table** from scratch!